

Implementation and Solution Document

Day 2 – From Counting to Predicting: TF-IDF to Distributional Embeddings

Submitted By:

Manahil

1. Task Overview

The objective of this project was to explore the evolution of text representation techniques from frequency-based methods to predictive word embeddings. The implementation included developing TF-IDF from scratch, validating it with Scikit-learn, building a cosine similarity retrieval system, training a Word2Vec Skip-gram model, and analyzing the strengths and limitations of both approaches through practical experiments.

2. Solution 1 – TF-IDF from Scratch

Objective

To manually implement the TF-IDF algorithm without using any machine learning library.

Implementation Steps

1. Loaded the cleaned corpus into Python.
2. Split the corpus into sentences and tokenized each sentence into individual words.
3. Built a vocabulary containing all unique words in the corpus.
4. Computed the Bag-of-Words representation for each sentence.
5. Calculated the Term Frequency (TF) for every word.
6. Calculated the Document Frequency (DF) and Inverse Document Frequency (IDF).
7. Multiplied TF and IDF values to obtain the final TF-IDF scores.
8. Saved the generated results for further analysis.

Observation

The implementation successfully generated TF-IDF scores for all documents. Frequently occurring words within a document received higher TF values, while common words across multiple documents obtained lower IDF values, reducing their overall importance.

Outcome

The manual implementation correctly converted textual information into weighted numerical vectors suitable for similarity analysis and information retrieval.

3. Solution 2 – Verification Using Scikit-learn

Objective

To verify the correctness of the manually implemented TF-IDF algorithm.

Implementation Steps

1. Imported TfidfVectorizer.
2. Applied the vectorizer to the same corpus.
3. Generated TF-IDF vectors automatically.
4. Compared the generated vocabulary and vector dimensions with the manual implementation.

Observation

The generated vocabulary size and TF-IDF vectors were consistent with the manually computed implementation.

Outcome

The comparison confirmed that the custom TF-IDF implementation produced accurate results.

4. Solution 3 – Cosine Similarity Retrieval

Objective

To retrieve the most relevant documents using TF-IDF vectors and cosine similarity.

Implementation Steps

1. Generated TF-IDF vectors for all documents.
2. Converted the input query into a TF-IDF vector.
3. Computed cosine similarity between the query and each document.
4. Ranked documents in descending order based on similarity scores.

Observation

Documents containing terms closely related to the query obtained higher similarity scores and appeared at the top of the ranking.

Outcome

The retrieval system successfully identified and ranked the most relevant documents.

5. Solution 4 – Synonym Similarity Analysis

Objective

To evaluate whether TF-IDF can recognize semantic similarity between synonymous words.

Implementation Steps

1. Selected five synonym or paraphrase word pairs from the corpus.
2. Calculated similarity using TF-IDF vectors.
3. Compared the obtained similarity scores.

Observation

Most synonym pairs produced very low similarity scores because TF-IDF only considers exact word occurrences and ignores semantic meaning.

Outcome

The experiment demonstrated that TF-IDF is effective for lexical matching but cannot capture semantic relationships.

6. Solution 5 – Word2Vec Skip-gram Training

Objective

To learn distributed word embeddings using predictive training.

Implementation Steps

1. Preprocessed the corpus into sentences.
2. Trained a Word2Vec Skip-gram model using Gensim.
3. Configured the model with:

- Vector Size = 100
- Window Size = 5
- Minimum Count = 1
- Epochs = 50

4. Saved the trained model for further evaluation.

Observation

The model successfully learned dense vector representations for all vocabulary words.

Outcome

The trained model generated semantic word embeddings suitable for similarity and analogy tasks.

7. Solution 6 – Word2Vec Evaluation

Objective

To evaluate the quality of the learned word embeddings.

Implementation Steps

1. Performed nearest-neighbor queries.
2. Calculated similarity between selected word pairs.
3. Conducted analogy testing using vector arithmetic.

Observation

Words appearing in similar contexts obtained similar vector representations, resulting in higher semantic similarity than TF-IDF.

Outcome

The experiment confirmed that Word2Vec captures semantic relationships more effectively than frequency-based methods.

8. Solution 7 – Polysemy Analysis

Objective

To examine whether Word2Vec generates different vectors for the same word used in different contexts.

Implementation Steps

1. Selected the polysemous word "**bank**".
2. Retrieved its embedding from the trained model.
3. Analyzed whether the vector changed across different meanings.

Output

```
# Load the trained model
model = Word2Vec.load("word2vec.model")

word = "bank"

print("Polysemy Test")
print("-" * 40)

if word in model.wv:
    print(f"Word: {word}")
    print("\nFirst 10 values of its embedding:")
    print(model.wv[word][:10])

    print("\nObservation:")
    print("Word2Vec stores only one vector for each word.")
    print("Therefore, if a word appeared in multiple meanings,")
    print("the same embedding would still be returned.")
else:
    print(f"'{word}' not found in the vocabulary.")

... Polysemy Test
-----
```

Observation

The model returned the same embedding for the word regardless of its meaning because Word2Vec stores only one vector per word.

Outcome

The experiment highlighted the limitation of static embeddings and motivated the need for contextual embeddings such as those used in BERT and GPT.

9. Overall Outcomes

The project successfully implemented TF-IDF from scratch, verified its correctness using Scikit-learn, and developed a document retrieval system based on cosine similarity. Word2Vec demonstrated a significant improvement in capturing semantic relationships between words. However, its inability to generate context-dependent embeddings revealed the limitations of static word representations and emphasized the importance of contextual language models.

10. Key Learnings

- Developed a complete TF-IDF implementation from scratch.
- Validated the implementation using Scikit-learn.
- Built a cosine similarity retrieval system.
- Observed the semantic limitations of TF-IDF.
- Trained a Word2Vec Skip-gram model using Gensim.
- Evaluated word embeddings through similarity and analogy tests.
- Understood the limitation of static embeddings for polysemous words.
- Learned why contextual embedding models such as GPT were developed.